AI Engineering Mastery Roadmap — .NET Developer Edition (via Microsoft Agent Framework)

Goal: Go from .NET developer → production AI Engineer, using Microsoft Agent Framework (MAF) as the spine of your learning.

How to use this doc: Work top to bottom. Each phase has *Concepts*, *Hands-on*, and a *Checkpoint project*. Don't move to the next phase until you can build the checkpoint project without copy-pasting from a tutorial.

> Note on MAF: it's Microsoft's unified, open-source agent SDK for .NET and Python — the successor combining Semantic Kernel's enterprise features (state, type safety, telemetry) with AutoGen's multi-agent abstractions, plus native graph-based workflows. It sits on top of `Microsoft.Extensions.AI` (the `IChatClient` abstraction). MAF reached 1.0 GA in April 2026. Repo: `microsoft/agent-framework`. Keep an eye on the package still evolving quickly — check docs before trusting older tutorials.

> **Your stack:** Gemini API key instead of Azure OpenAI/OpenAI. This works fine — MAF only needs an `IChatClient` implementation, and Gemini has community adapters: `GeminiDotnet.Extensions.AI` (recommended, actively maintained) or `Mscc.GenerativeAI.Microsoft`. Expect minor friction (e.g. Gemini maps roles as `user`/`model` instead of `user`/`assistant`/`system` under the hood — the adapter handles this, but it's worth understanding why). No Azure subscription is required anywhere in this roadmap.

Phase 0 — Environment & Mental Model (3–5 days)

Concepts

- What an LLM actually does (next-token prediction, context window, tokens, temperature/top-p) — don't skip this, it demystifies "hallucination" and "prompt engineering."
- Difference between: a raw model API call, a "chat completion," an "agent," and a "workflow."
- The .NET AI stack layering: `Microsoft.Extensions.AI` (`IChatClient`, model-neutral) → `Microsoft.Agents.AI` (MAF, agent-level abstractions) → Azure AI Foundry (hosting/observability/eval).

Hands-on

- Install .NET 9 SDK, VS Code or Visual Studio, and get a Gemini API key from Google AI Studio.

- `dotnet add package Microsoft.Extensions.AI` and `dotnet add package GeminiDotnet.Extensions.AI`
- Write a 20-line console app that calls `IChatClient.GetResponseAsync` directly against Gemini — no agent, no framework magic. Understand what's happening before you abstract it away.

Checkpoint: A console app that sends a prompt to an LLM and streams the response token-by-token to the console.

Phase 1 — Core LLM App-Building Skills (1–2 weeks)

Concepts

- Prompt engineering fundamentals: system vs user messages, few-shot examples, structured output (JSON mode / schema-constrained generation).
- Function/tool calling — the mechanism, not just the syntax: how the model decides to call a tool, what gets sent back, why this is the foundation of agents.
- Conversation state & context management — why context windows fill up and how truncation/summarization strategies work.
- Embeddings and vector similarity (conceptually — you'll use it in Phase 3).

Hands-on

- `dotnet add package Microsoft.Agents.AI`
- Build your first `AIAgent` with instructions and one custom tool (a C# method decorated so MAF can call it — e.g., a "get weather" or "look up order status" function).
- Force structured JSON output from the model and deserialize it into a C# record.

Checkpoint: A CLI assistant agent with 2–3 tools (e.g., a "personal assistant" that can check the time, do a calculation, and look up a fact from a local dictionary) that maintains conversation history across turns using MAF's agent session/thread abstraction.

Phase 2 — MAF Core Architecture (2 weeks)

Concepts

- `IAIAgent` / `AIAgent` and `ChatClientAgent` — the central abstractions.
- Agent sessions/threads: serialize → run → deserialize pattern for multi-turn persistence.
- Context providers — how agent "memory" is injected (short-term vs long-term).

- Middleware — intercepting agent actions (logging, guardrails, cost tracking, redaction).
- Dependency injection: registering `IChatClient` and agents in ASP.NET Core's service container.

Hands-on

- Refactor your Phase 1 project into an ASP.NET Core Web API where the agent is DI-registered.
- Add middleware that logs every tool call and every token count.
- Persist agent sessions to a real store (start with a file or SQLite, not in-memory) so conversations survive a restart.

Checkpoint: A web API with a `/chat` endpoint backed by a MAF agent, with persisted conversation state and at least one piece of custom middleware.

Phase 3 — Retrieval-Augmented Generation (RAG) (2 weeks)

Concepts

- Why RAG exists: grounding responses in your own data, reducing hallucination, working around context-window limits.
- Chunking strategies, embedding models, vector stores (Azure AI Search, Qdrant, or a simple in-memory vector store for learning).
- MAF's `TextSearchProvider` (an AI Context Provider) pattern — attaching a search adapter to an agent so retrieved snippets get injected automatically or via a tool call.
- Citation handling and answer grounding.

Hands-on

- Ingest a small document set (e.g., your own resume + a few PDFs) into a vector store.
- Wire a `TextSearchProvider` into your Phase 2 agent so it can answer questions using retrieved context, with citations.

Checkpoint: A "chat with your docs" agent that answers questions only from ingested documents and cites its sources.

Phase 4 — Multi-Agent Systems & Workflows (2–3 weeks)

Concepts

- When to use a single agent with tools vs. a multi-agent workflow.
- MAF's graph-based workflows: explicit control over multi-agent execution paths (sequential, parallel, conditional branching, human-in-the-loop approval gates).
- Orchestration patterns: planner/executor, critic/reviewer loops, specialist agents (e.g., a "researcher" agent handing off to a "writer" agent).
- State management across long-running, multi-step workflows.

Hands-on

- Build a 2-agent workflow: a "researcher" agent that gathers info + tools, and a "writer" agent that turns findings into a report — explicitly wired as a MAF workflow graph, not just prompt-chaining.
- Add a human-in-the-loop approval step before the workflow takes an irreversible action (e.g., "send email").

Checkpoint: A multi-agent workflow that solves a real task end-to-end (e.g., research a topic, draft a summary, and pause for your approval before "publishing").

Phase 5 — Tool Ecosystem: MCP, A2A, and External Integration (1–2 weeks)

Concepts

- Model Context Protocol (MCP): what it standardizes, why it matters vs. hand-rolled tool integrations. MCP is GA in MAF 1.0.
- Connecting MAF agents to MCP servers as tool sources.
- Agent-to-Agent (A2A) protocol — currently preview; know what it's for even if you don't build with it yet.
- Security basics for tool-using agents: treat tool/content inputs as untrusted, prompt-injection risk, least-privilege tool design.

Hands-on

- Connect your agent to an existing MCP server (there are public/sample ones) instead of a hand-written tool.
- Add basic guardrails: validate tool inputs, restrict what a tool is allowed to touch.

Checkpoint: An agent that uses at least one MCP-sourced tool, with input validation on that tool.

Phase 6 — Production Engineering (2–3 weeks)

Concepts

- Observability: OpenTelemetry traces from MAF — for this roadmap, use a self-hosted OTel backend (Jaeger or the .NET Aspire dashboard) rather than Application Insights, since you're not on Azure.
- Evaluation: how to test agent quality, not just "did it run" — golden datasets, LLM-as-judge patterns, regression testing prompts.
- Cost and token tracking, latency budgeting, streaming UX.
- **Containerization**: packaging your MAF agent API into a Docker image — this is your deployment target for this roadmap, not a specific cloud's proprietary hosting.
- Secrets management: environment variables / platform secret stores for your Gemini API key — never baked into the image or committed to source control.
- CodeAct pattern (advanced/optional): collapsing many small tool-call round-trips into one generated program run in a sandbox — useful once you feel orchestration overhead becoming a bottleneck.

Hands-on

- Wire OpenTelemetry into your Phase 4 workflow and view traces via Jaeger or the Aspire dashboard, running locally in Docker.
- Write an evaluation script that runs your agent against 10–20 fixed test prompts and scores the outputs.
- Write a multi-stage `Dockerfile` for your ASP.NET Core agent API. Build and run it locally with `docker run`, confirming behavior matches your dev environment with the API key injected via environment variable.

Checkpoint: Your agent API running as a Docker container locally, with full request tracing, a basic eval suite, and no secrets in source control.

Phase 7 — Capstone Project (2–4 weeks)

Pick one and build it fully, production-style (auth, observability, tests, README):

1. **Domain assistant with RAG + tools** — e.g., an internal support agent that answers from your company/product docs and can perform 2–3 real actions (create a ticket, check an order).
2. **Multi-agent content pipeline** — research → draft → review/critique loop → human approval → publish, with full tracing.

3. **Agentic ops tool** — an agent with shell/filesystem access (agent harness pattern) that can triage logs or run diagnostics with human-in-the-loop approval for destructive actions.

Ship it with: DI-registered agents, persisted sessions, MCP tool integration, OpenTelemetry, an eval suite, and a short write-up of design tradeoffs.

Deployment (required): Ship the capstone as a Docker container to a free/low-cost host — **Fly.io** or **Render** (both deploy straight from a Dockerfile, free tier is enough for a portfolio project), or a cheap VPS (DigitalOcean, Hetzner) if you want raw `docker run` + reverse-proxy experience. This is the core, transferable skill: a containerized service deploys identically anywhere, which matters more at this stage than picking a specific cloud.

Deployment (optional stretch goal, after the roadmap): Once the container runs cleanly, redeploy the *same image* to:

- **Azure Container Apps or App Service** — worth doing once since MAF is a Microsoft framework; you'll see the "native" OpenTelemetry → Application Insights pipeline the official docs assume, which helps if you target .NET/Microsoft-stack employers.
- **AWS App Runner or ECS Fargate** — shows you're not locked to one vendor, useful if you target companies outside the Microsoft ecosystem.

Each redeploy should take half a day or less once the Docker image exists — treat it as proof of portability, not a place to relearn the whole roadmap.

Ongoing / Parallel Track (don't block on these, but revisit weekly)

- Read the MAF migration guides if you have Semantic Kernel or AutoGen background — they highlight *why* MAF made certain design choices, which deepens understanding faster than tutorials alone.
- Follow MAF's GitHub Discussions/Discord and weekly office hours — the framework moves fast; tutorials go stale.
- Once comfortable, skim LangChain/LangGraph (Python) conceptually — not to switch stacks, but because most AI engineering discourse and papers use that vocabulary, and MAF's workflow graphs map closely to LangGraph's ideas.

Suggested pace

- Full-time focus: ~10–12 weeks total.
- Evenings/weekends (5–8 hrs/week): ~5–6 months.

A note on rigor

Treat "AI Engineering" as: (1) solid software engineering fundamentals you already have, (2) LLM-specific concepts (Phase 0–1), (3) an agent framework as tooling (Phases 2–5), (4) production discipline (Phase 6) that most tutorials skip and most jobs actually require. Employers filter hard on #4 — prioritize it once you're past the basics rather than endlessly tutorial-hopping on #2–3.